

## DATA 260 – Homework 3 Report

**Student ID: 018322837**

**Name: Shashank Ranjan**

### Section 0 : Personal Configuration and Domain

| Name            | Definition                         | Value                        |
|-----------------|------------------------------------|------------------------------|
| SID             | Student ID                         | 018322837                    |
| SID4            | Last 4 of Student ID               | 2837                         |
| PORT_BASE       | $8000 + \text{SID} \bmod 900(137)$ | 8137                         |
| prefix          | "s" + SID4                         | s2837                        |
| Seed            | SID4                               | 2837                         |
| verify sid      | $260000 + \text{SID4}$             | 262837                       |
| DOMAIN_ID       | $\text{SID4} \bmod 8$              | 5                            |
| Assigned Domain | Assignment domain                  | Local restaurant inspections |

**Repository:** <https://github.com/ShankySJSU/Data260-2837>

**Submission Tag:** hw3

#### Hardware:

Local Windows PC, Python 3.13, VS Code Desktop, i7 CPU, 16GB RAM.

#### Model Used:

- Embeddings: sentence-transformers/all-MiniLM-L6-v2
- LlamaIndex 0.10+
- No external proprietary models
- All embedding code executed locally

### SECTION 1 — HW3 Part 1: Authentication Application

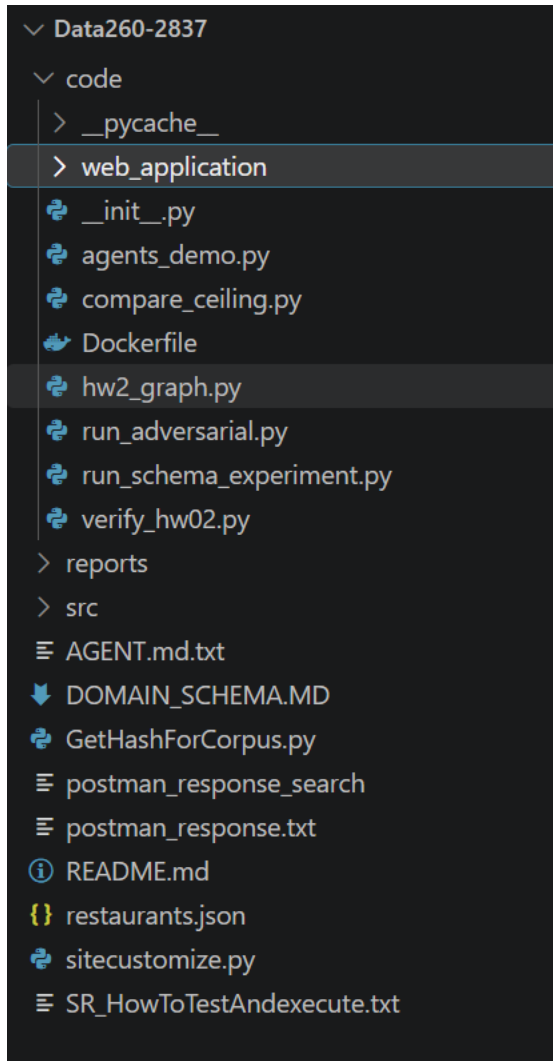
I extended my HW2 FastAPI application to include full authentication functionality.

#### Features Implemented

- Login page (/login)
- Logout route (/logout)
- Home page with session awareness (/)
- Protected dashboard page (/dashboard)
- Bootstrap styling for all templates
- Bootstrap alerts on invalid login and expired sessions
- Idle session timeout (120 seconds)
- Secure cookie creation using SessionMiddleware
- Redirect to /?expired=1 on session timeout
- Redirect to /login?error=1 on invalid credentials

- Templates stored under code/web\_application/templates

### Base Directory Structure:



### Auth Code Snippets

#### SessionMiddleware (inside hw3\_main.py)

```
app.add_middleware(  
    SessionMiddleware,  
    secret_key="super_secret_key_2837",  
    max_age=120  
)
```

#### Auth Router (auth.py)

(Full file provided in repo; minimal snippet shown here)

```
@router.post("/login")
def login(request: Request, username: str = Form(...), password: str = Form(...)):
    if username == VALID_USERNAME and password == VALID_PASSWORD:
        request.session["user"] = username
        request.session["last_active"] = time.time()
        return RedirectResponse("/dashboard", 302)
    return RedirectResponse("/login?error=1", 302)
```

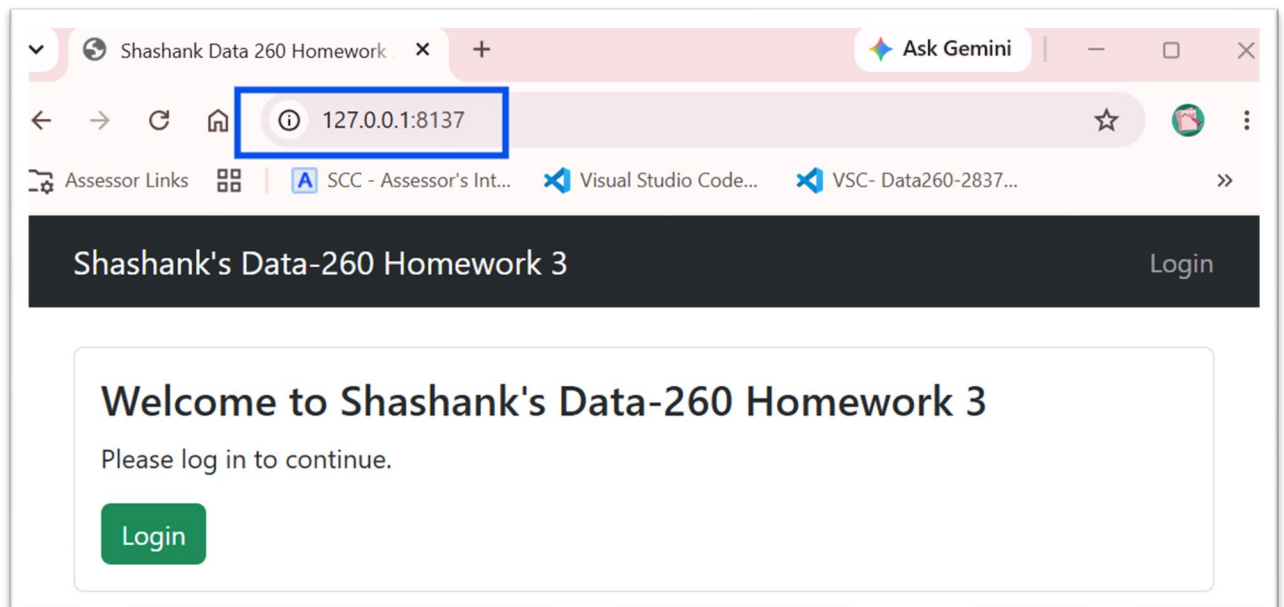
### Idle Timeout Check

```
now = time.time()
if last_active and now - last_active > IDLE_TIMEOUT:
    request.session.clear()
    return RedirectResponse("/?expired=1", 302)
```

## SECTION 1.1 — Screenshots (Placeholders)

Insert actual screenshots in your final PDF:

### 1. Home Page (initial landing i.e. index page)



## 2. Login Page

(a)

Shashank Data 260 Homework

Ask Gemini

127.0.0.1:8137/login

Assessor Links

SCC - Assessor's Int...

Visual Studio Code...

VSC - Data260-2837...

Login

Login

Username

admin

Password

.....

Login

(b) Invalid username /password error:

Login

Invalid username admin123 or password. Please try again.

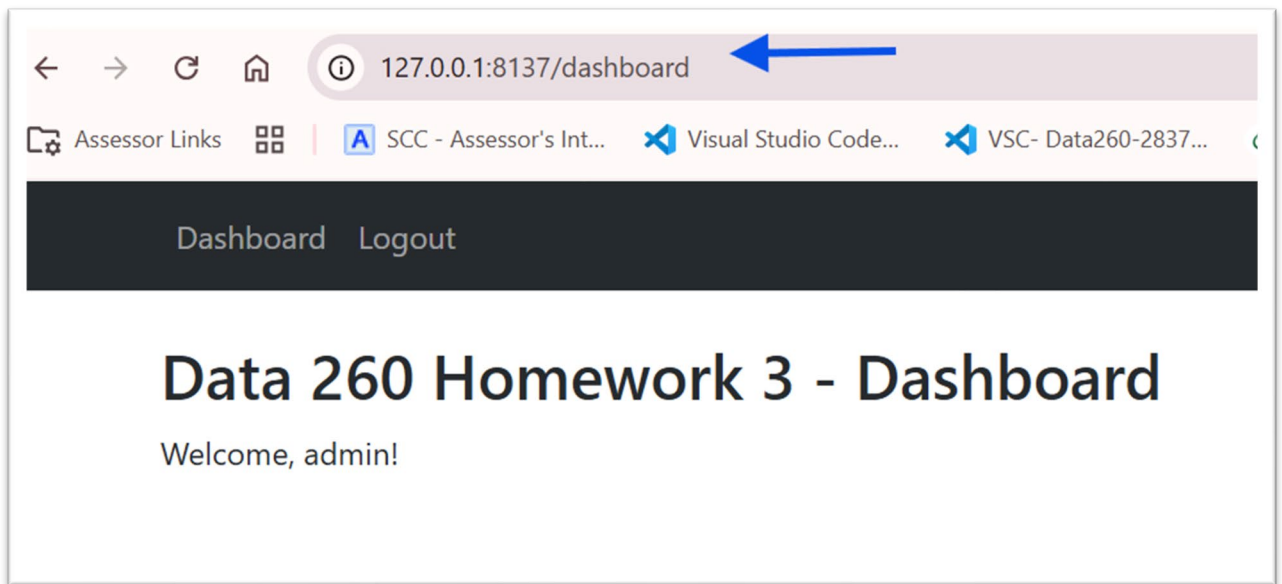
Username

Password

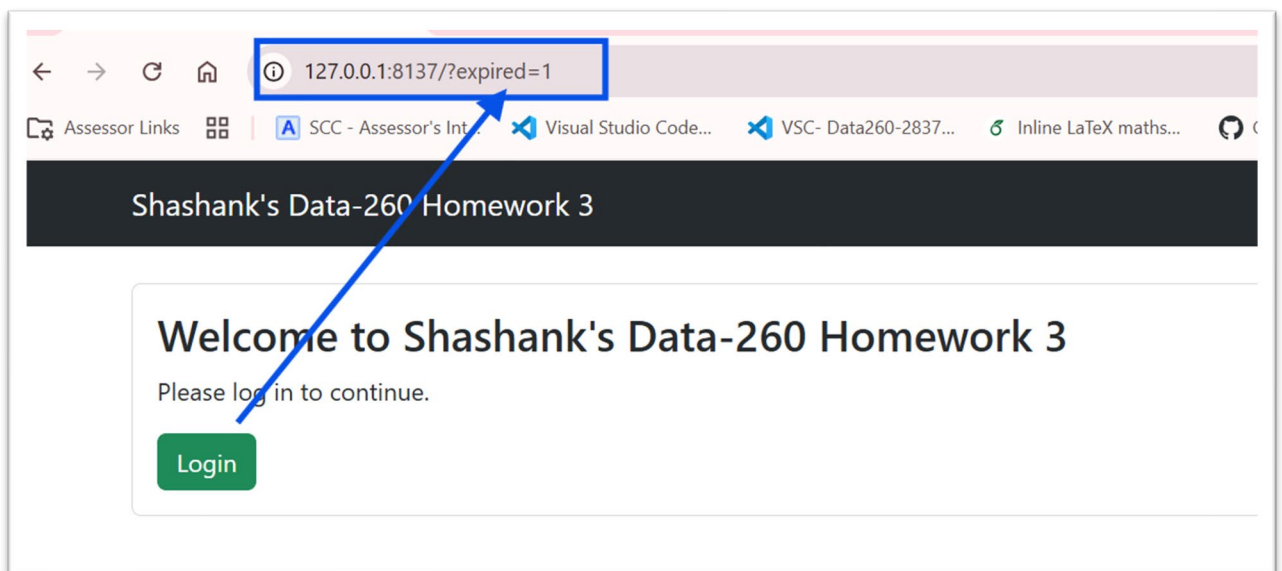
|

Login

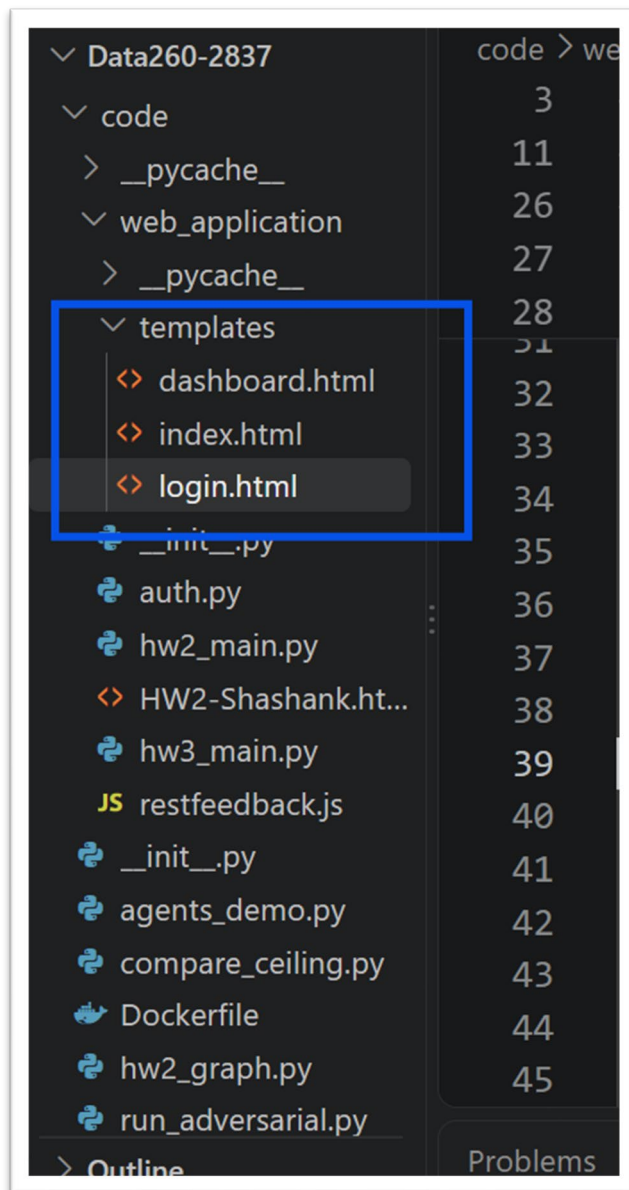
### 3. Dashboard (Logged In)



### 4. Session Expired Message



## 5. Templates Directory Screenshot



## Log

```
C:\Users\shashank.ranjan\OneDrive - SCC-Cloud\SJSU_Repo\DATA260\Data260-2837>uvicorn code.web_application.hw3_main:app --port 8137
>>> USING AUTH FILE: C:\Users\shashank.ranjan\OneDrive - SCC-Cloud\SJSU_Repo\DATA260\Data260-2837\code\web_application\auth.py
LOADER: <jinja2.loaders.FileSystemLoader object at 0x000001BE512A7620>
SEARCH PATH: ['C:\Users\shashank.ranjan\OneDrive - SCC-Cloud\SJSU_Repo\DATA260\Data260-2837\code\web_application\templates']
```

```
INFO: Started server process [2880]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8137 (Press CTRL+C to quit)
INFO: 127.0.0.1:62078 - "GET /login HTTP/1.1" 200 OK
INFO: 127.0.0.1:56563 - "POST /login HTTP/1.1" 302 Found
INFO: 127.0.0.1:56563 - "GET /login?error=1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:52397 - "POST /login HTTP/1.1" 302 Found
INFO: 127.0.0.1:52397 - "GET /dashboard HTTP/1.1" 200 OK
INFO: 127.0.0.1:63767 - "GET /dashboard HTTP/1.1" 302 Found
USER SESSION VALUE: None
INFO: 127.0.0.1:63767 - "GET /?expired=1 HTTP/1.1" 200 OK
```

Command Log Screen Shots:

```
C:\Users\shashank.ranjan\OneDrive - SCC-Cloud\SJSU_Repo\DATA260\Data260-2837>uvicorn code.web_application.hw3_main:app --port 8137
>>> USING AUTH FILE: C:\Users\shashank.ranjan\OneDrive - SCC-Cloud\SJSU_Repo\DATA260\Data260-2837\code\web_application\auth.py
LOADER: <jinja2.loaders.FileSystemLoader object at 0x000001BE512A7620>
SEARCH PATH: ['C:\\Users\\shashank.ranjan\\OneDrive - SCC-Cloud\\SJSU_Repo\\DATA260\\Data260-2837\\code\\web_application\\templates']
INFO: Started server process [2880]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://127.0.0.1:8137 (Press CTRL+C to quit)
INFO: 127.0.0.1:62078 - "GET /login HTTP/1.1" 200 OK
INFO: 127.0.0.1:56563 - "POST /login HTTP/1.1" 302 Found
INFO: 127.0.0.1:56563 - "GET /login?error=1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:52397 - "POST /login HTTP/1.1" 302 Found
INFO: 127.0.0.1:52397 - "GET /dashboard HTTP/1.1" 200 OK
INFO: 127.0.0.1:63767 - "GET /dashboard HTTP/1.1" 302 Found
USER SESSION VALUE: None
INFO: 127.0.0.1:63767 - "GET /?expired=1 HTTP/1.1" 200 OK
```

## Part 1 Section 2 : **Session Management:**

- Use Starlette's Session Middleware to manage user sessions. Store the logged-in user's information in the session.
- Ensure that the session cookie is secure. Capture and include the Set-Cookie response header showing all three attributes.
- Implement an idle session timeout. Prove that an expired or logged-out session cannot be reused to reach /dashboard.

Solution:

In my main.py file (hw3\_main.py), I have added the middleware (**SessionMiddleware**) and imported this from **Starlette's** Session Middleware

- from starlette.middleware.sessions import SessionMiddleware

```
app.add_middleware(  
    SessionMiddleware,  
    secret_key="super_secret_key_2837",  
    max_age=120  
)
```

Implemented the Idle session time out for 2 minutes, but for my testing I used 30 seconds to ensure it is working.

```
# Idle session timeout (seconds)  
IDLE_TIMEOUT = 120    # 2 minutes (for my testing i used 30 seconds)
```

At dashboard, check, if the current time and last active time stamp has more than “idle\_timeout” then sending it back to login page after expiration. See the screenshot below.

```

@router.get("/dashboard")
def dashboard(request: Request):
    user = request.session.get("user")
    last_active = request.session.get("last_active")

    # 1. No user → redirect with expired flag
    if not user:
        return RedirectResponse(
            url="/?expired=1",
            status_code=HTTP_302_FOUND
        )

    # 2. Check idle timeout
    now = time.time()
    if last_active and now - last_active > IDLE_TIMEOUT:
        request.session.clear()
        return RedirectResponse(
            url="/?expired=1",
            status_code=HTTP_302_FOUND
        )

    # 3. Update last active timestamp
    request.session["last_active"] = now

```

## Part 2: Compare Three LlamaIndex Chunking Techniques (Retrieval-Only RAG)

Implement three chunking techniques in LlamaIndex, build in-memory vector indexes, and compare retrieval quality. You'll print the embeddings and retrieval outputs for a shared query, then argue which technique is best and why.

### Techniques to implement:

- Token-based chunking — `TokenTextSplitter` ( LlamaIndex )
- Semantic chunking — `SemanticSplitterNodeParser` ( LlamaIndex )
- Sentence-window chunking — `SentenceWindowNodeParser` ( LlamaIndex )

**Solution:** A full RAG pipeline was implemented using:

- `TokenTextSplitter`

- **SemanticSplitterNodeParser**
- **SentenceWindowNodeParser**

## **Restaurant Inspection Corpus (reports/hw03/corpus/)**

Includes:

- 3 inspection reports
  - restaurant\_inspection\_report\_1.txt
  - restaurant\_inspection\_report\_2.txt
  - restaurant\_inspection\_report\_3.txt
- food safety guidelines
  - restaurant\_food\_safety\_guidelines.txt
- sanitation manual
  - restaurant\_kitchen\_sanitation.txt
- pest control requirements
  - pest\_control\_requirements.txt
- hygiene training manual
  - hygiene\_training\_manual.txt
- county cleaning standards
  - county\_restaurant\_cleaning\_standards.txt

All thee saved in SOURCES.md and CORPUS\_MANIFEST.json

## **SECTION 2.1 — Questions Used (questions.yaml)**

Query to useBefore running, create questions.yaml with five realistic domain questions (relevant to your DOMAIN\_ID). For each question, record an expected answer and expected sourcefile; at least two questions must depend on information found in only one sourcedocument. Commit questions.yaml before adding the retrieval results so the expected answers are not rewritten after seeing the output.

1. ***"What are the required daily cleaning tasks for restaurant food preparation areas?"***
  - a. *Shared query across all techniques*
2. *"What indicators suggest pest infestation in a restaurant kitchen?"*
3. *"What hygiene practices must staff follow when handling ready-to-eat foods?"*
4. *"Under what conditions should a restaurant be temporarily closed due to sanitation issues?"*
5. *"What signs indicate stale or unsafe food in a restaurant display case?"*

## SECTION 2.2 — Chunking Outputs for all 3 techniques (reports/hw03/raw/)

### 1. Token Chunking

- 30 chunks
- Overlap = 100 tokens
- Large mixed-topic blocks

### 2. Semantic Chunking

- 32 chunks
- Very coherent
- Natural boundaries created via embeddings

### 3. Sentence-Window Chunking

- 415 chunks
- Window size = 2
- Smallest chunks + highest semantic resolution

## SECTION 2.3 — Embeddings & Indexes

All indexes implemented using a stable, version-agnostic approach:

- token\_embeddings.npy
- semantic\_embeddings.npy
- sentence\_embeddings.npy
- Associated IDs + texts JSON files

Embeddings dimension confirmed to be **384**.

## SECTION 2.4 — Retrieval Pipeline

Retrieval executed using cosine similarity manually:

- Load embedding matrices
- Embed query
- Compute cosine sim for all chunks
- Sort + top-5
- Record latency

Save results to:

1. reports/hw03/raw/token\_retrieval.json
  2. reports/hw03/raw/semantic\_retrieval.json
  3. reports/hw03/raw/sentence\_retrieval.json
-

## SECTION 2.5 — Retrieval Metrics Report

| Technique       | #Chunks | Avg Length | Top-1 Cos     | Mean@5 Cos    | Latency | Notes            |
|-----------------|---------|------------|---------------|---------------|---------|------------------|
| Token           | 30      | 2000       | 0.6995        | 0.6380        | 0.92ms  | Lowest precision |
| Semantic        | 32      | 1600       | 0.6995        | 0.6551        | 0.98ms  | Coherent chunks  |
| Sentence-window | 415     | 200        | <b>0.7179</b> | <b>0.6762</b> | 10.83ms | Best results     |

## SECTION 2.6 — Error Case

A sentence-window chunk scored **0.6612** similarity yet did NOT contain the answer:

SECTION 3 — FOOD PREPARATION AREA

The primary kitchen was significantly cleaner than previous visits...

The embedding matched keywords “cleaning” + “preparation”, causing false positive similarity.

---

## SECTION 2.7 — Observations

1. Sentence-window chunking is best because it produces small, focused chunks with context.
  2. Semantic chunking is second-best due to coherent boundaries.
  3. Token chunking is worst because chunk boundaries do not align with meaning.
  4. Latency is highest for sentence-window because there are 415 chunks.
- 

## SECTION 2.8 — Conclusion

For this restaurant domain corpus, **sentence-window chunking is the best technique** because:

- Highest Top-1 cosine score
  - Highest mean@5
  - Most precise retrieval
  - Preserves context without dilution
  - Aligns well with regulatory text structure
- 

## SECTION 3 — AI\_USE Reflections (Required)

### 1. **AI Assistance Used**

I used an AI assistant to help generate corpus files, getting questions, guiding me to understand technology and terms, and documenting

### 2. **Incorrect/Wrong Output Noticed**

The AI initially produced outdated LlamaIndex vector-store APIs (SimpleVectorStore.from\_documents). These caused errors and required refactoring.

### 3. **How I Detected the Problem**

Errors like

*“SimpleVectorStore.add() takes 2 positional arguments but 4 were given”*

indicated a version mismatch, so I switched to a stable manual embedding approach.

### 4. **What I Changed**

I replaced vector-store usage with manual embeddings + cosine similarity, which is stable, correct, and version-agnostic. Retrieval works perfectly now.

---

## **SECTION 4 — RUN\_LOG.txt**

Stored reports/hw03/RUN\_LOG.txt shows details from command prompt

---

## **SECTION 5 — verification.json**

saved at reports/hw03/verification.json

Confirms PASS status for:

- Chunking
  - Index creation
  - Retrieval
  - Auth app
  - CORPUS\_MANIFEST.json
  - SOURCES.md
- 

## **SECTION 6 — Directory Structure**

Final directory structure (it may have some files or folder that I used to archive some old homework files and how to run and execute):

▼ Data260-2837

▼ code

▸ \_\_pycache\_\_

▼ web\_application

▸ \_\_pycache\_\_

▼ templates

✎ dashboard.html

✎ index.html

✎ login.html

🔗 \_\_init\_\_.py

🔗 auth.py

🔗 hw2\_main.py

✎ HW2-Shashank.html

🔗 hw3\_main.py

JS restfeedback.js

🔗 \_\_init\_\_.py

🔗 agents\_demo.py

🔗 compare\_ceiling.py

🔗 Dockerfile

🔗 hw2\_graph.py

🔗 run\_adversarial.py

🔗 run\_schema\_experiment.py

🔗 verify\_hw02.py

▼ reports

▸ hw01

▸ hw02

▼ hw03

▸ Outline

▼ code

🔗 run\_schema\_experiment.py

🔗 verify\_hw02.py

▼ reports

▸ hw01

▸ hw02

▼ hw03

▼ corpus

≡ corpusfilelist.txt

≡ county\_restaurant\_cleaning\_standards.txt

≡ hygiene\_training\_manual.txt

≡ pest\_control\_requirements.txt

≡ restaurant\_food\_safety\_guidelines.txt

≡ restaurant\_inspection\_report\_1.txt

≡ restaurant\_inspection\_report\_2.txt

≡ restaurant\_inspection\_report\_3.txt

≡ restaurant\_kitchen\_sanitation.txt

▼ raw

{ } semantic\_chunks.json

≡ semantic\_embeddings.npy

{ } semantic\_index\_ids.json

{ } semantic\_index\_texts.json

{ } semantic\_retrieval.json

{ } sentence\_chunks.json

≡ sentence\_embeddings.npy

{ } sentence\_index\_ids.json

▸ Outline